

Docker in CI/CD

A pipeline turns a commit into an image, proves the image is good, and puts it somewhere a server can pull it from. Everything below is about doing that so the result is reproducible and can be undone.

1. Setting it up

A pipeline needs three things: a runner that can build images, a registry to push to, and credentials that let it in.

The runner is the part with a choice to make.

Approach	How it works	Cost
Mounted socket	The host's <code>/var/run/docker.sock</code> is given to the job	Fast. The job is root on the host
Docker-in-Docker	A privileged <code>dind</code> service beside the job	Isolated. Slower, and needs privileged mode
Managed builder	The platform runs BuildKit for us	Simplest. The usual choice on hosted CI

On GitHub Actions the managed builder is the default path, and the whole setup is three steps:

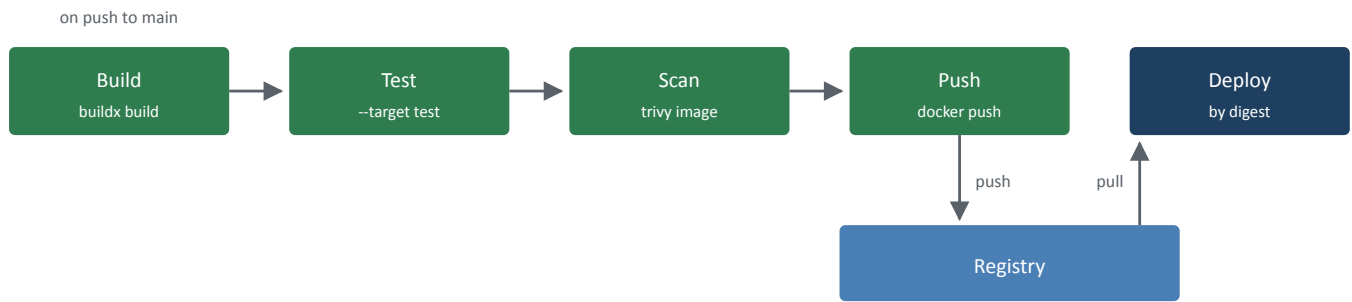
```
- uses: docker/setup-buildx-action@v3

- uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }

- uses: docker/build-push-action@v6
  with:
    push: true
    tags: ghcr.io/acme/payroll:${ github.sha }
```

Prefer short-lived credentials. A registry password stored as a secret works and never expires, which is the problem. OIDC lets the pipeline exchange its own identity for a token that lasts minutes.

```
permissions:
  contents: read
  packages: write
  id-token: write
```



Rollback is the same pull arrow, pointed at an older digest. That is the whole reason the registry sits between the pipeline and the servers.

2. Building images in CI

The build itself is the same `docker build` from Part 3, with two differences that matter.

The cache starts cold. A hosted runner is a fresh machine, so nothing is reused unless the pipeline arranges it. Section 6 covers that.

Tests belong in the build, not after it. A multi-stage file can run them as a stage, so a failing test fails the build and no image is produced.

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B package -DskipTests

FROM build AS test
RUN mvn -B verify

FROM eclipse-temurin:21-jre-alpine AS runtime
WORKDIR /app
COPY --from=build /src/target/ payroll.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
- name: Test
  uses: docker/build-push-action@v6
  with:
    target: test
    push: false

- name: Build
  uses: docker/build-push-action@v6
  with:
    target: runtime
    push: true
```

Build for the architecture that will run it. A runner is usually `amd64`, but a developer on an M-series Mac is not, so state the platform rather than inheriting it.

```
platforms: linux/amd64,linux/arm64
```

3. Image tagging strategy

Every build must produce an image with a name nothing else will ever have. That is the whole requirement; the rest is convenience.

Tag	Example	When
Commit sha	<code>payroll:sha-9f2c1ab</code>	Every build, always
Semantic version	<code>payroll:1.4.2</code>	On a release tag
Moving minor	<code>payroll:1.4</code>	On a release tag, moves
Branch	<code>payroll:pr-482</code>	Pull request previews
<code>latest</code>	<code>payroll:latest</code>	Only from the default branch

```
- uses: docker/metadata-action@v5
  id: meta
  with:
    images: ghcr.io/acme/payroll
    tags: |
      type=sha,prefix=sha-
      type=semver,pattern={{version}}
      type=semver,pattern={{major}}.{{minor}}
      type=ref,event=pr
      type=raw,value=latest,enable={{is_default_branch}}
```

The sha tag is the one that matters. It is the only tag guaranteed to name exactly one build, so it is what a deployment records and what a rollback names. Everything else exists so that people can read the list.

4. Pushing images to a registry

```
- uses: docker/build-push-action@v6
  with:
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    labels: ${{ steps.meta.outputs.labels }}
```

A push uploads only the layers the registry does not already have. A rebuild that changed one application layer uploads a few megabytes, not the whole image.

Capture the digest. The tags are how people refer to the image; the digest is how the deployment should.

```
- id: build
  uses: docker/build-push-action@v6
  with:
    push: true
    tags: ${{ steps.meta.outputs.tags }}

- run: echo "Deployed digest ${{ steps.build.outputs.digest }}"
```

Registry	Fits
GitHub Container Registry	Code already on GitHub, token already there
Amazon ECR	Deploying to ECS or EKS
Google Artifact Registry	Deploying to GKE or Cloud Run
Harbor, Nexus	Self hosted, air-gapped, or strict retention rules

A pipeline must not pull anonymously from Docker Hub. The rate limit is per address, and a shared CI address reaches it quickly. Authenticate, or mirror the base images into the private registry.

5. Automated image scanning

A scanner compares what is installed in the image against known vulnerability data. It covers two things at once: the operating system packages in the base image, and the application dependencies inside the jar.

```
- uses: aquasecurity/trivy-action@0.28.0
  with:
    image-ref: ghcr.io/acme/payroll:sha-${{ github.sha }}
    severity: HIGH,CRITICAL
    exit-code: '1'
    ignore-unfixed: true
```

`exit-code: 1` is what makes it a gate rather than a report. Without it the scan prints findings and the pipeline carries on.

`ignore-unfixed` keeps the gate usable. A vulnerability with no released fix cannot be acted on, and blocking every build on it teaches everybody to skip the scan.

Scan after the push, against the pushed image. Scanning a locally built image proves something about a file that no longer exists; scanning the pushed one proves something about what will be deployed.

A pass is a statement about today. The image does not change, but the vulnerability data does, so an image that passed last month may not pass now. Two habits follow: scan the registry on a schedule, not only at build time, and rebuild when the base image publishes a fix.

Generating a software bill of materials at the same time records what was in the image, which is what makes the next question answerable in minutes:

```
- uses: anchore/sbom-action@v0
  with:
    image: ghcr.io/acme/payroll:sha-${{ github.sha }}
```

6. Build caching

A cold cache means every build downloads every dependency. The fix is to keep the cache somewhere outside the runner.

```
- uses: docker/build-push-action@v6
  with:
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

Cache type	Stored in	Use when
<code>type=gha</code>	The Actions cache	On GitHub Actions
<code>type=registry</code>	A tag in the registry	Any CI, and shared across runners
<code>type=inline</code>	The image itself	Simple, but only caches the final stage
<code>type=local</code>	A directory	A self-hosted runner with a persistent disk

```
cache-from: type=registry,ref=ghcr.io/acme/payroll:buildcache
cache-to: type=registry,ref=ghcr.io/acme/payroll:buildcache,mode=max
```

`mode=max` caches every stage, not just the last. Without it the build stage of a multi-stage file is not cached, which is the stage that downloads Maven dependencies and takes the time.

Layer ordering still matters more than any cache backend. A Dockerfile that copies the source before resolving dependencies has nothing worth caching, and no cache configuration rescues it.

7. Deploying images

Deployment means telling something to run a specific image. What that something is changes the command and nothing else.

Target	The deploy step
A single server	<code>docker compose pull && docker compose up -d</code>
Kubernetes	<code>kubectl set image</code> or a manifest change
Amazon ECS	Register a new task definition and update the service
Cloud Run	<code>gcloud run deploy --image</code>

```
docker compose pull
docker compose up -d --no-build
```

`--no-build` is deliberate. The deploy step must never build. If it does, the thing tested is not the thing running, and the pipeline's guarantees are gone.

Deploy the digest, not the tag.

```
IMAGE=ghcr.io/acme/payroll@sha256:4a1c8f3b9e2d7c6a5b4f3e2d1c0b9a8f
kubectl set image deployment/payroll payroll=$IMAGE
```

A tag can be moved between the moment the pipeline reads it and the moment the server pulls it. A digest cannot, so two servers deploying the same digest are running the same bytes, and that is provable rather than assumed.

8. Immutable deployments

Build the image once and promote that exact image through every environment. Staging and production run the same digest. Nothing is rebuilt on the way.

```
build → image@sha256:4a1c8f... → dev → staging → production
                                   (the same digest at each step)
```

The alternative, a build per environment, looks harmless and breaks the guarantee: the staging build and the production build happened at different times, resolved dependencies separately, and are not the same software. Testing one says little about the other.

What makes this possible is keeping configuration out of the image. The test is simple: if the same image cannot run in two environments without being rebuilt, something that belongs in configuration was baked into it.

```
# wrong: this image can only ever run against staging
ENV SPRING_DATASOURCE_URL=jdbc:postgresql://staging-db:5432/payroll
ENV API_KEY=sk_live_9f2c1ab
```

The second line is worse than the first. A secret written into a layer stays in the image and in the registry, whatever the running container is told later.

In the image	Injected at run time
The application and its dependencies	Database URLs and hostnames
The JVM and the base packages	Credentials and API keys
The entrypoint	Feature flags, log levels, profiles

```
environment:
  SPRING_PROFILES_ACTIVE: production
  SPRING_DATASOURCE_URL: jdbc:postgresql://db.internal:5432/payroll
```

Nothing in a running container should be edited. A fix applied with `docker exec` disappears at the next restart and leaves the running system different from the image everybody is reading. Change the source, build a new image, deploy it.

9. Rollback using previous image versions

A rollback is a deploy of the previous digest. There is nothing to rebuild, because the previous image still exists.

```
kubectl rollout undo deployment/payroll # the previous revision
kubectl rollout undo deployment/payroll --to-revision=4
```

```
IMAGE=ghcr.io/acme/payroll@sha256:2b8d41c7e9a0f3e5d7c9a1b3e5f7a9c1
docker compose pull && docker compose up -d
```

Three things have to be true beforehand, and all three are easy to get wrong:

The old image must still be in the registry. A retention policy that deletes images older than seven days also deletes the thing being rolled back to. Keep every image a production environment has run, and prune only branch and preview tags.

The deployed digest must be recorded. "Roll back to what was running yesterday" needs an answer, and neither `latest` nor a git commit message is one. Write the digest into the deployment record.

The database must tolerate it. Rolling the application back does not roll back a migration. Schema changes should be compatible with the previous version, which in practice means adding columns before using them and dropping them a release later.

A rollback is faster than a fix. Restoring service takes a minute, while a forward fix takes as long as a build, a test run and a deploy. Roll back first and diagnose afterwards.

10. The pipeline together

```
name: build
on:
  push:
    branches: [main]
    tags: ['v*']

permissions:
  contents: read
  packages: write

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${github.actor}
          password: ${secrets.GITHUB_TOKEN}
```

```
      - uses: docker/metadata-action@v5
        id: meta
        with:
          images: ghcr.io/acme/ payroll
          tags: |
            type=sha,prefix=sha-
            type=semver,pattern={{version}}
            type=raw,value=latest,enable={{is_default_branch}}
```

Then the two builds. The first proves the tests pass and produces nothing; the second is the image that ships.

```

- uses: docker/build-push-action@v6
  with:
    target: test
    push: false

- uses: docker/build-push-action@v6
  id: build
  with:
    target: runtime
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    cache-from: type=gha
    cache-to: type=gha,mode=max

```

And the gate, pointed at the digest the push returned rather than at a tag:

```

- uses: aquasecurity/trivy-action@0.28.0
  with:
    image-ref: ghcr.io/acme/payroll@${{ steps.build.outputs.digest }}
    severity: HIGH,CRITICAL
    exit-code: '1'
    ignore-unfixed: true

```

Step	Why it is there
<code>setup-buildx</code>	BuildKit, which the cache backends need
<code>login</code>	Credentials, scoped to this run
<code>metadata</code>	Tags derived from the branch, sha and release tag
<code>build --target test</code>	Tests fail the build before an image exists
<code>build --target runtime</code>	The image that ships, pushed with its tags
<code>cache-from</code> / <code>cache-to</code>	The cold-cache fix
<code>trivy</code>	The gate, against the pushed digest

For example, a commit lands on `main`. The pipeline builds the test stage and it passes, builds the runtime stage and pushes it as `sha-9f2c1ab` and `latest`, and scans the pushed digest. The deploy step sets the Kubernetes image to that digest. Two days later an error rate rises, the previous digest is read out of the deployment record, and `kubectrl rollout undo` restores service in under a minute while the cause is still being looked for.

The habit worth building is naming the digest everywhere it can be named. Tags are for people, digests are for machines, and every hard deployment question turns out to be a question about which bytes were running.